

variable, but that variable might never be set in the code of class `D`. In cases like this, you must look for the instance variable in the code of class `C`. If it is not found there, then you must look in the code of class `B`, and so on.

When designing a class hierarchy, perhaps the best way to avoid this problem is to only call methods and use instance variables inherited from one layer up in the hierarchy. In our example, code in class `D` should only make calls to methods in class `C`, while class `C` should only make calls to methods in class `B`, and so on. This is a simplistic version of the *Law of Demeter*. Stated simply, you (meaning objects) should only talk to your immediate friends (nearby objects) and never talk to strangers (distant objects). A detailed discussion is beyond the scope of this book, but there are many references available on the internet.

Another approach, which we first talked about in Chapter 4, is to take advantage of *composition*, where an object instantiates one or more other objects. The key difference is that inheritance is used to model an “is a” relationship, whereas composition uses a “has a” relationship. For example, if we wanted to have a spinbox widget (an editable text number field with an up and a down arrow), we could build a `SpinBox` class that instantiates a `DisplayNumber` object and two `CustomButton` objects for the arrows. Each of these objects already knows how to handle its user interactions.

MULTIPLE INHERITANCE

You’ve seen how a class can inherit from another class. In fact, Python (like some other programming languages) allows a class to inherit from more than just one class. This is known as *multiple inheritance*. The Python syntax for inheriting from more than one class is quite straightforward:

```
class SomeClass(<BaseClass1>, <BaseClass2>, ...):
```

However, it’s important to be aware that multiple inheritance can potentially introduce conflicts when the base classes you inherit from contain identically named methods and/or instance variables. Python does have rules (known as the *method resolution order*, or *MRO*) to resolve these potential problems. I consider this an advanced topic and will not cover it here, but if you want to look into it, a detailed discussion can be found at <https://www.python.org/download/releases/2.3/mro>.